Multi – level Linked List

–Destroy List(Main List and Child)

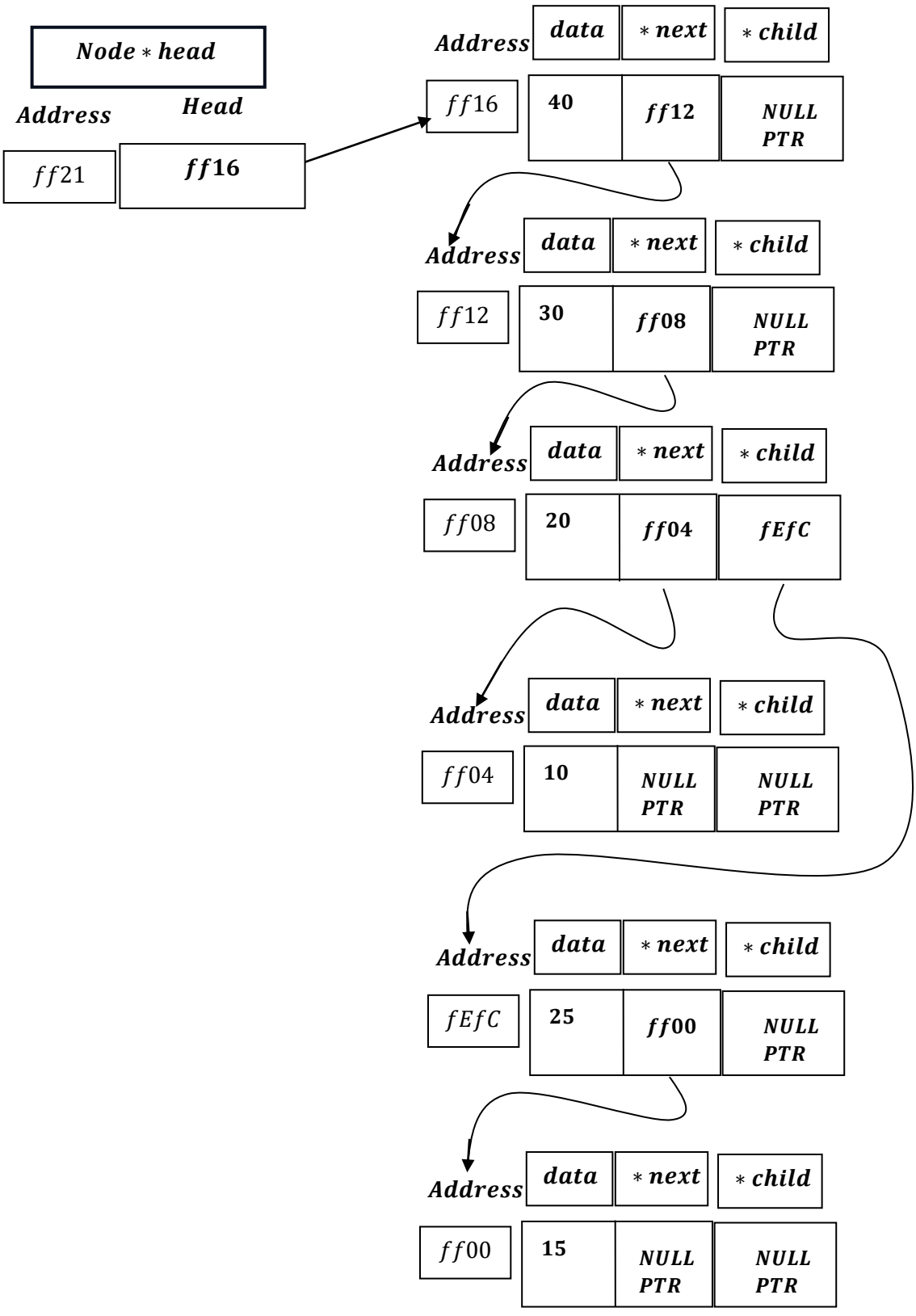
–Program Analysis

Program:

```
void destroyNodeRecursively(Node *node)
{
    if (node == nullptr)
        return;

    // 1. Destroy Child List first
    destroyNodeRecursively(node->child);

    // 2. Destroy Next List
    destroyNodeRecursively(node->next);

    // 3. Free current
    cout << "Freeing node: " << node->data << endl;
    free(node);
}

void destroyList()
{
    destroyNodeRecursively(head);
    head = nullptr;
    cout << "Entire list destroyed." << endl;
}
int main()
{
    int choice;
    while (true)
    {
        cout << "\n--- Multilevel List Menu ---" << endl;
        cout << "8. Destroy List" << endl;
       cin >> choice;

        switch (choice)
```

```
        {
        case 8:
            destroyList();
            break;
        default:
            cout << "Invalid choice" << endl;
        }
    }
    return 0;
}
```

Program Analysis



***As, it is, recursion*:**

***Base Case*:**

```
    if (node == nullptr)
        return;
```

***As soon as destory list function is called*:**

```
        case 8:
            destroyList();
            break;
```

```
void destroyList()
{
    destroyNodeRecursively(head);
    head = nullptr;
    cout << "Entire list destroyed." << endl;
}
```

Then inside destroyList function ,

destroyNodeRecursively(head: ff16) , is called.

destroyNodeRecursively(head: ff16)
destroyList()

```
    void destroyNodeRecursively(Node *node)
```

Here Node * node = Head = ff16.

```
    if (node == nullptr)
        return;
```

$Node \neq Nullptr$, *hence skips*.

```
    destroyNodeRecursively(node->child);
```

$ff16 \rightarrow child = NULLPTR$.

$\rightarrow$ *PUSH NULLPTR to the STACK*.

$destroyNodeRecursively(node: ff16 \rightarrow Child \coloneqq NULLPTR) \rightarrow Stack\ Frame$
$destroyNodeRecursively(head: ff16) \rightarrow Stack\ Frame$

Now , it will assign, as it call recursively :

$destroyNodeRecursively(Node * node) =$

$destroyNodeRecursively(node \rightarrow child: NULLPTR\ )$

$i.e. Node * node = node \rightarrow child: NULLPTR$

And, it is the base case:

```
    if (node == nullptr)
        return;
```

As it goes to base case , return ; hence pop the current stack frame for nullptr pop up immediately .

$destroyNodeRecursively(node: ff16 \rightarrow Child: NULLPTR) \rightarrow Stack\ Frame$
$destroyNodeRecursively(head: ff16) \rightarrow Stack\ Frame$

POPPED

And, it will exit the current function call.

$destroyNodeRecursively(Node * node: NULLPTR)$

$doesn't\ exit\ from\ the\ function[As, it\ is\ recursive]$

```
    destroyNodeRecursively(node->next);
```

$destroyNodeRecursively(node: ff16 \rightarrow next: ff12)$

$\rightarrow PUSH\ ff12\ to\ the\ stack.$

$destroyNodeRecursively(node: ff16 \rightarrow NEXT := ff12) \rightarrow Stack\ Frame$
$destroyNodeRecursively(head: ff16) \rightarrow Stack\ Frame$

Now , it will assign, as it call recursively :

$destroyNodeRecursively(Node * node) =$

$destroyNodeRecursively(node \rightarrow Next: ff12)$

$And\ , now\ , node \neq NULLPTR$

Again, it will run InOrder:

```
    destroyNodeRecursively(node->child);
```

$destroyNodeRecursively(node \rightarrow child)$

$\Rightarrow destroyNodeRecursively(node: ff12 \rightarrow child: NULLPTR)$

$\rightarrow PUSH\ NULLPTR\ to\ the\ STACK.$

$destroyNodeRecursively(node: ff12 \rightarrow child := NULLPTR) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff16 \rightarrow NEXT := ff12) \rightarrow Stack\ Frame$
$destroyNodeRecursively(head := ff16) \rightarrow Stack\ Frame$

Now , it will assign, as it call recursively :

$destroyNodeRecursively(Node * node) =$

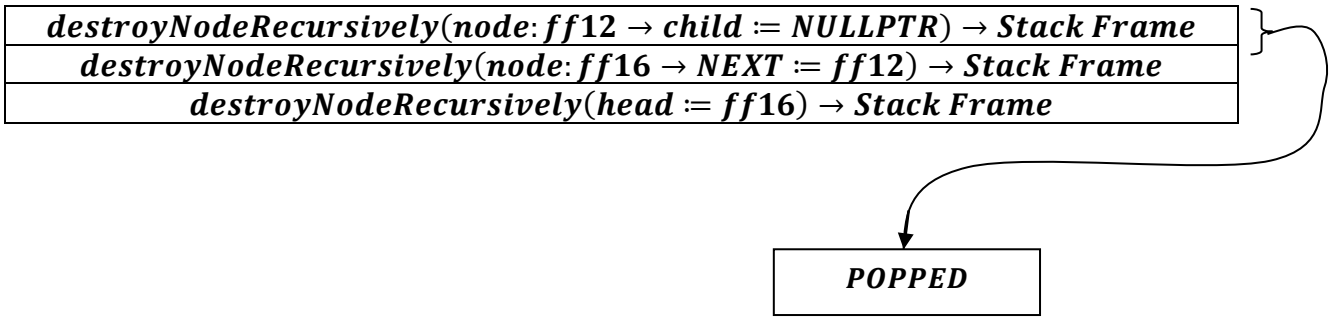
$destroyNodeRecursively(node \rightarrow child: NULLPTR)$

$i.e. Node * node = node \rightarrow child: NULLPTR$

$And, it\ is\ the\ base\ case:$

```
    if (node == nullptr)
        return;
```

$As\ it\ goes\ to\ base\ case\ , return\ ; hence\ pop\ the\ current\ stack\ frame\ for\ nullptr\ pop\ up\ immediately\ .$



$And, it\ will\ exit\ the\ current\ function\ call.$

$destroyNodeRecursively(Node * node: NULLPTR\ )$

$doesn't\ exit\ from\ the\ function[As, it\ is\ recursive].$

```
    destroyNodeRecursively(node->next);
```

$destroyNodeRecursively(node: ff12 \rightarrow next: ff08)$

$\rightarrow PUSH\ ff08\ to\ the\ STACK.$

$destroyNodeRecursively(node: ff12 \rightarrow Next := ff08) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff16 \rightarrow NEXT := ff12) \rightarrow Stack\ Frame$
$destroyNodeRecursively(head := ff16) \rightarrow Stack\ Frame$

$Now\ , it\ will\ assign, as\ it\ call\ recursively\ :$

$destroyNodeRecursively(Node * node) =$

$destroyNodeRecursively(node \rightarrow Next: ff08\ )$

$$And\ , now\ , node \neq NULLPTR$$

$$Again, it\ will\ run\ InOrder:$$

```
    destroyNodeRecursively(node->child);
```

$$destroyNodeRecursively(node \rightarrow child)$$

$$\Rightarrow destroyNodeRecursively(node: ff08 \rightarrow child: fEfC)$$

$$\rightarrow PUSH\ fEfC\ to\ the\ STACK.$$

$destroyNodeRecursively(node: ff08 \rightarrow Child \coloneqq fEfC) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff12 \rightarrow Next \coloneqq ff08) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff16 \rightarrow NEXT \coloneqq ff12) \rightarrow Stack\ Frame$
$destroyNodeRecursively(head \coloneqq ff16) \rightarrow Stack\ Frame$

$$destroyNodeRecursively(Node * node) =$$

$$destroyNodeRecursively(node \rightarrow child: fEfC\ )$$

$$And\ , now\ , node \neq NULLPTR$$

$$Again, it\ will\ run\ InOrder \begin{bmatrix} here\ , node \rightarrow child\ recursively\ called \\ again, as\ base\ case\ is\ not\ reached. \end{bmatrix}:$$

```
    destroyNodeRecursively(node->child);
```

$$if\ node \rightarrow child \neq NULLPTR,$$

$$destroyNodeRecursively(node \rightarrow child\ ) keeps\ calling$$

$$recursively\ until\ the\ base\ case: node = NULLPTR\ is\ met.$$

```
    destroyNodeRecursively(node->child);
```

$$destroyNodeRecursively(node \rightarrow child)$$

$\Rightarrow destroyNodeRecursively(node: fEfC \rightarrow child: NULLPTR)$

$\rightarrow PUSH\ NULLPTR\ to\ the\ STACK.$

$destroyNodeRecursively(node: fEfC \rightarrow Child \coloneqq NULLPTR) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff08 \rightarrow Child \coloneqq fEfC) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff12 \rightarrow Next \coloneqq ff08) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff16 \rightarrow NEXT \coloneqq ff12) \rightarrow Stack\ Frame$
$destroyNodeRecursively(head \coloneqq ff16) \rightarrow Stack\ Frame$

Now , it will assign, as it call recursively :

$destroyNodeRecursively(Node * node) =$

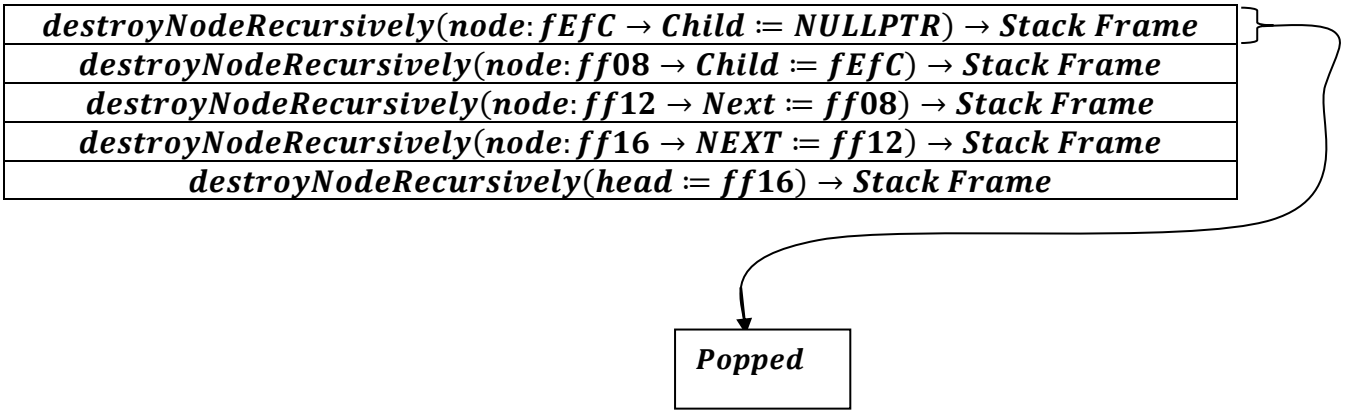
$destroyNodeRecursively(node \rightarrow child: NULLPTR\ )$

$i.e. Node * node = node \rightarrow child: NULLPTR$

And, it is the base case:

```
    if (node == nullptr)
        return;
```

As it goes to base case , return ; hence pop the current stack frame for nullptr pop up immediately .



And, it will exit the current function call.

$destroyNodeRecursively(Node * node: NULLPTR\ )$

$doesn't\ exit\ from\ the\ function[As, it\ is\ recursive].$

```
    destroyNodeRecursively(node->next);
```

$destroyNodeRecursively(node: fEfC \rightarrow next: ff00)$

$\rightarrow PUSH\ ff00\ to\ the\ STACK.$

$destroyNodeRecursively(node: fEfC \rightarrow NEXT \coloneqq ff00) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff08 \rightarrow Child \coloneqq fEfC) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff12 \rightarrow Next \coloneqq ff08) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff16 \rightarrow NEXT \coloneqq ff12) \rightarrow Stack\ Frame$
$destroyNodeRecursively(head \coloneqq ff16) \rightarrow Stack\ Frame$

$Again, it\ will\ run\ InOrder:$

```
    destroyNodeRecursively(node->child);
```

$destroyNodeRecursively(node \rightarrow child)$

$\Rightarrow destroyNodeRecursively(node: ff00 \rightarrow child: NULLPTR)$

$\rightarrow PUSH\ NULLPTR\ to\ the\ STACK.$

$destroyNodeRecursively(node: ff00 \rightarrow Child \coloneqq NULLPTR) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: fEfC \rightarrow NEXT \coloneqq ff00) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff08 \rightarrow Child \coloneqq fEfC) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff12 \rightarrow Next \coloneqq ff08) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff16 \rightarrow NEXT \coloneqq ff12) \rightarrow Stack\ Frame$
$destroyNodeRecursively(head \coloneqq ff16) \rightarrow Stack\ Frame$

$Now\ ,it\ will\ assign, as\ it\ call\ recursively:$

$destroyNodeRecursively(Node * node) =$

$destroyNodeRecursively(node \rightarrow child: NULLPTR)$

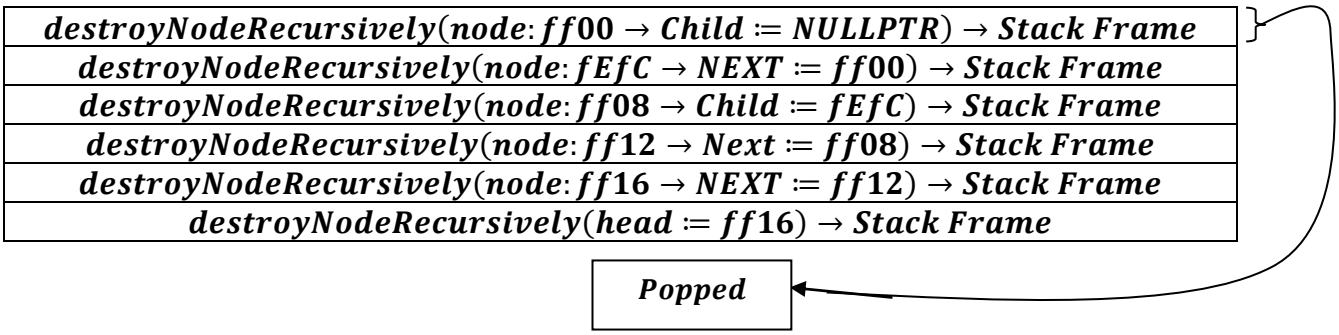
$i.e. Node * node = node \rightarrow child: NULLPTR$

$And, it\ is\ the\ base\ case:$

```
    if (node == nullptr)
        return;
```

$As\ it\ goes\ to\ base\ case\ ,return\ ;hence\ pop\ the\ current\ stack$

frame for nullptr pop up immediately .



And, it will exit the current function call.

*destroyNodeRecursively(Node * node: NULLPTR)*

doesn't exit from the function[As, it is recursive].

```
    destroyNodeRecursively(node->next);
```

destroyNodeRecursively(node: ff00 → NEXT: NULLPTR)

→ PUSH NULLPTR to the STACK.

destroyNodeRecursively(node: ff00 → NEXT := NULLPTR) → Stack Frame
destroyNodeRecursively(node: fEfC → NEXT := ff00) → Stack Frame
destroyNodeRecursively(node: ff08 → Child := fEfC) → Stack Frame
destroyNodeRecursively(node: ff12 → Next := ff08) → Stack Frame
destroyNodeRecursively(node: ff16 → NEXT := ff12) → Stack Frame
destroyNodeRecursively(head := ff16) → Stack Frame

Now , it will assign, as it call recursively :

*destroyNodeRecursively(Node * node) =*

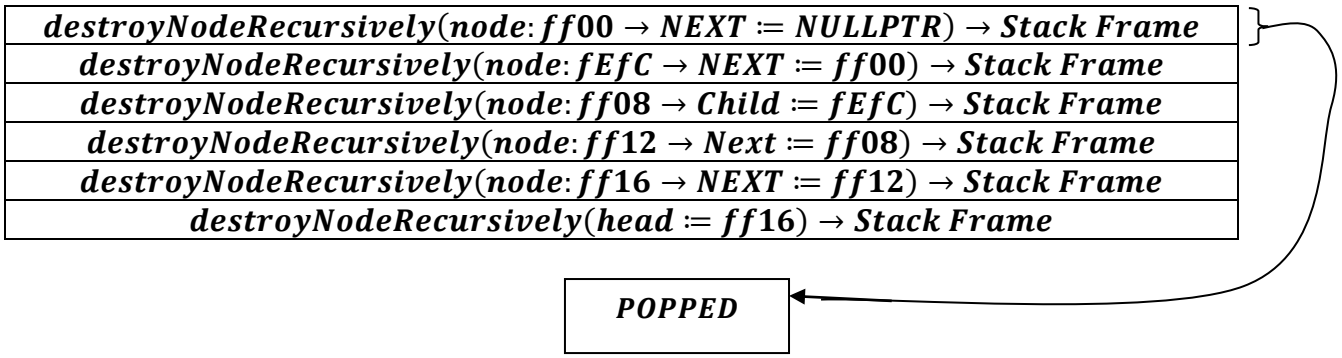
destroyNodeRecursively(node → child: NULLPTR)

*i.e. Node * node = node → child: NULLPTR*

And, it is the base case:

```
    if (node == nullptr)
        return;
```

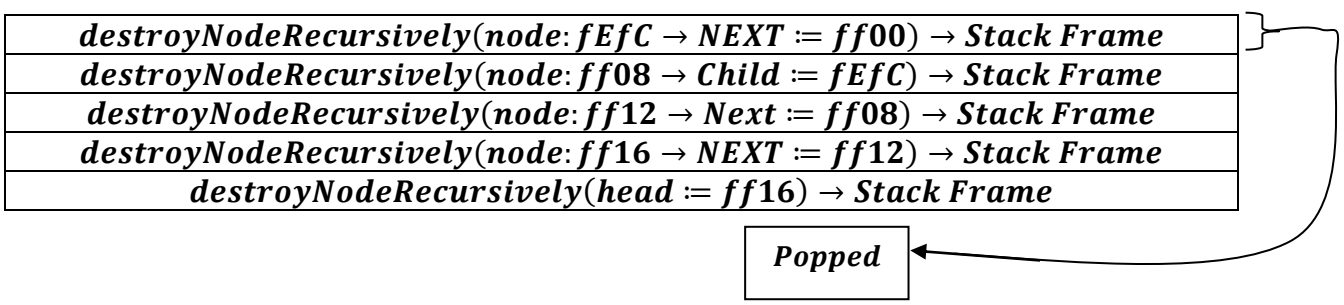
As it goes to base case , return ; hence pop the current stack frame for nullptr pop up immediately .



So both recursive calls return immediately $\left[\begin{array}{c}\textbf{Base Case Met}\\ \textbf{for both the calls}\\ \textbf{i.e. NULLPTR}\end{array}\right]$.

```
    // 1. Destroy Child List first
    destroyNodeRecursively(node->child);

    // 2. Destroy Next List
    destroyNodeRecursively(node->next);
```

This is the time , when the deeper recursive calls finish , Now it ready for unwinding.

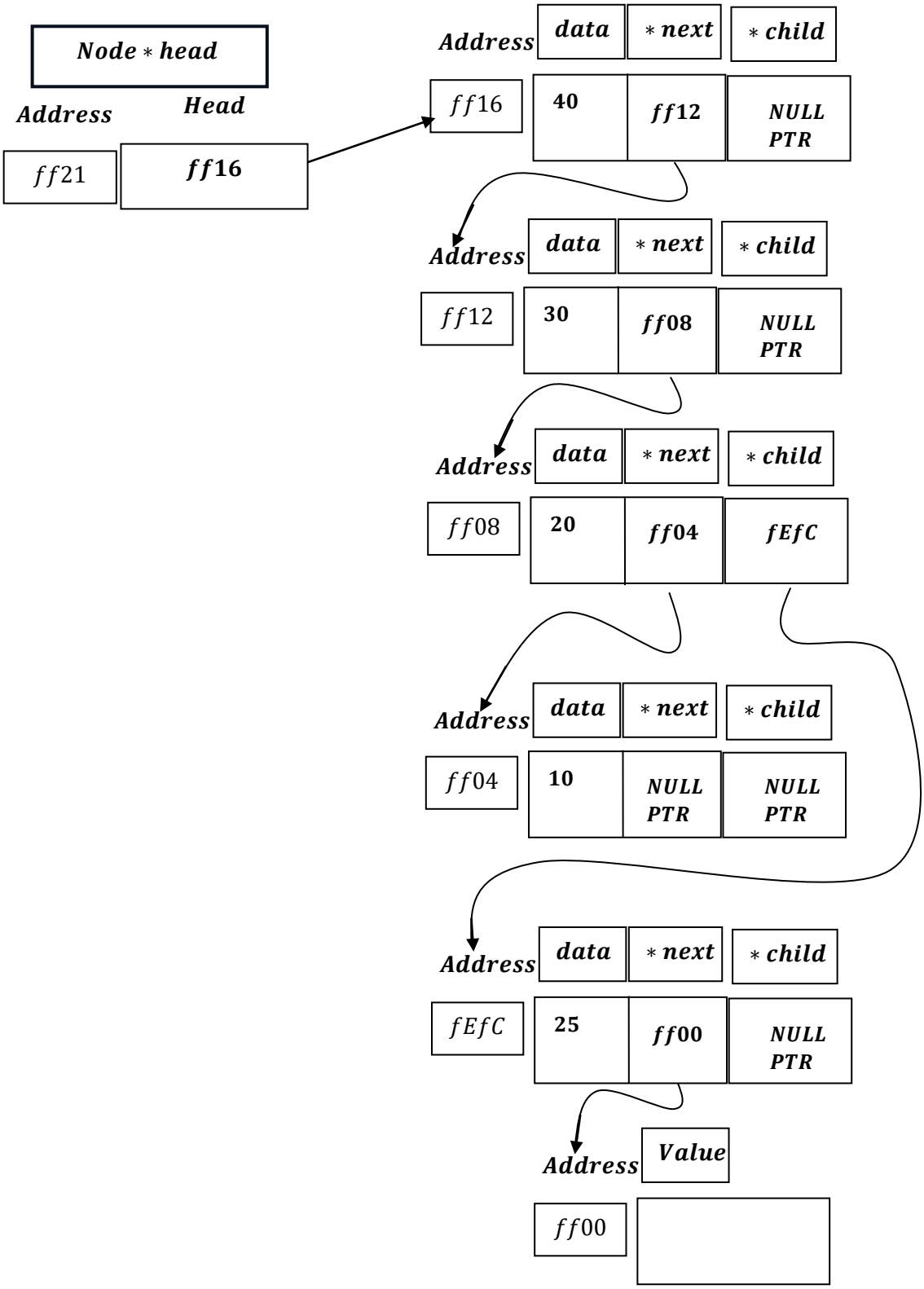


And it will run:

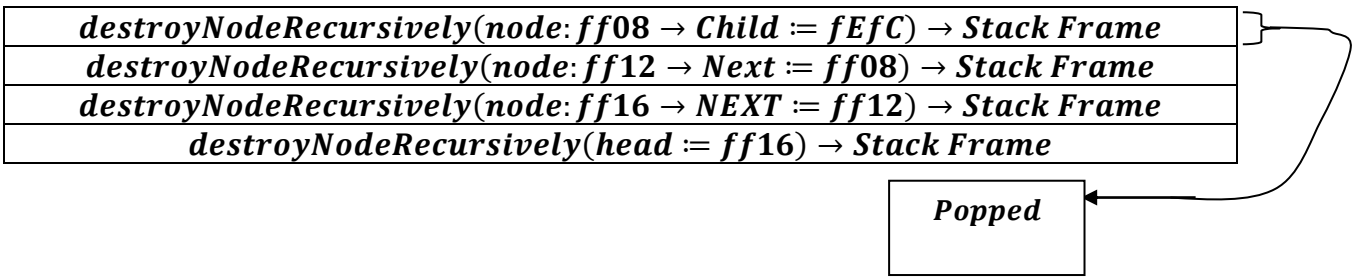
```
    cout << "Freeing node: " << node->data << endl;
    free(node);
```

$\boldsymbol{Print: ``Freeing\ Node:``\ [node: ff00 \rightarrow data] = 15}$.

$\boldsymbol{Free(node: ff00)}$



Next:

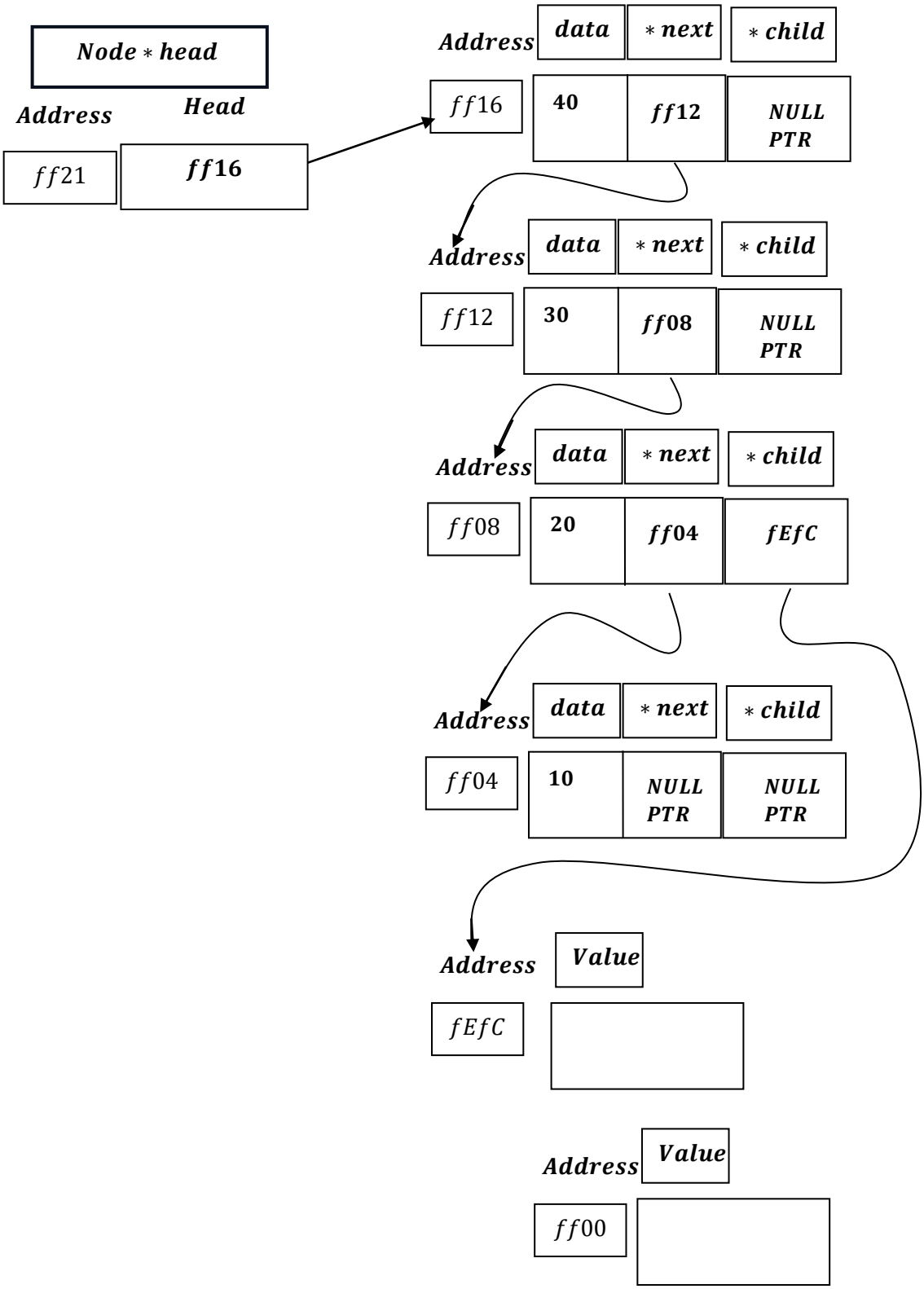


And it will run:

```
    cout << "Freeing node: " << node->data << endl;
    free(node);
```

Print : ``***Freeing Node***:`` [***node***: ***fEfC*** → ***data***] = **25** .

Free(***node***: ***fEfC***)


Node * head
Address
Head
ff21
ff16
Address
data
* next
* child
ff16
40
ff12
NULL PTR
Address
data
* next
* child
ff12
30
ff08
NULL PTR
Address
data
* next
* child
ff08
20
ff04
fEfC
Address
data
* next
* child
ff04
10
NULL PTR
NULL PTR
Address
Value
fEfC
Address
Value
ff00

Now we stand here:

$destroyNodeRecursively(node: ff12 \rightarrow Next \coloneqq ff08) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff16 \rightarrow NEXT \coloneqq ff12) \rightarrow Stack\ Frame$
$destroyNodeRecursively(head \coloneqq ff16) \rightarrow Stack\ Frame$

Now it will backtrack to :

$\Rightarrow destroyNodeRecursively(node: ff08 \rightarrow Next: ff04)$

As, Compiler already created instruction for the program stored in code − segment part of the memory.

Now each recursive call stack creates a stack frame ,
consisting of :

→ ***Return Address***
→ ***Parameters***
→ ***local variables***
→ ***saved state***.

During Back Tracking :
→ ***CPU pops return address from the stack***.
→ ***Then load it to Program Counter(PC)***.

Then its the work of Program Counter(PC):

$\rightarrow$ ***It jumps back to the return address*** $(\boldsymbol{say}: \mathbf{1005})$, ***The return address of the Code Segment of the Memory***.

$\rightarrow$ ***And, then it loads the instruction from the Code Segment from address*** $(\mathbf{1005})$.

Here the instruction is to execute:

$\Rightarrow \boldsymbol{destroyNodeRecursively}(\boldsymbol{node}: \boldsymbol{ff08} \rightarrow \boldsymbol{Next}: \boldsymbol{ff04})$

$\boldsymbol{destroyNodeRecursively}(\boldsymbol{node}: \boldsymbol{ff08} \rightarrow \boldsymbol{next}: \boldsymbol{ff04})$

$\rightarrow$ ***PUSH*** $\boldsymbol{ff04}$ ***to the STACK***.

$\boldsymbol{destroyNodeRecursively}(\boldsymbol{node}: \boldsymbol{ff08} \rightarrow \boldsymbol{Next} := \boldsymbol{ff04}) \rightarrow \boldsymbol{Stack\ Frame}$
$\boldsymbol{destroyNodeRecursively}(\boldsymbol{node}: \boldsymbol{ff12} \rightarrow \boldsymbol{Next} := \boldsymbol{ff08}) \rightarrow \boldsymbol{Stack\ Frame}$
$\boldsymbol{destroyNodeRecursively}(\boldsymbol{node}: \boldsymbol{ff16} \rightarrow \boldsymbol{NEXT} := \boldsymbol{ff12}) \rightarrow \boldsymbol{Stack\ Frame}$
$\boldsymbol{destroyNodeRecursively}(\boldsymbol{head} := \boldsymbol{ff16}) \rightarrow \boldsymbol{Stack\ Frame}$

Now , it will assign, as it call recursively :

$\boldsymbol{destroyNodeRecursively}(\boldsymbol{Node} * \boldsymbol{node}) =$

$\boldsymbol{destroyNodeRecursively}(\boldsymbol{node} \rightarrow \boldsymbol{Next}: \boldsymbol{ff04})$

And , now , node $\neq$ ***NULLPTR***

Again, it will run InOrder:

```
    destroyNodeRecursively(node->child);
```

$\Rightarrow \boldsymbol{destroyNodeRecursively}(\boldsymbol{node}: \boldsymbol{ff04} \rightarrow \boldsymbol{child}: \boldsymbol{NULLPTR})$

$\rightarrow$ ***PUSH NULLPTR to the STACK.***

$destroyNodeRecursively(node: ff04 \rightarrow Child := NULLPTR) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff08 \rightarrow Next := ff04) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff12 \rightarrow Next := ff08) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff16 \rightarrow NEXT := ff12) \rightarrow Stack\ Frame$
$destroyNodeRecursively(head := ff16) \rightarrow Stack\ Frame$

Now , it will assign, as it call recursively :

$destroyNodeRecursively(Node * node) =$

$destroyNodeRecursively(node \rightarrow child: NULLPTR)$

$i.e. Node * node = node \rightarrow child: NULLPTR$

And, it is the base case:

```
    if (node == nullptr)
        return;
```

As it goes to base case , return ; hence pop the current stack frame for nullptr pop up immediately .

$destroyNodeRecursively(node: ff04 \rightarrow Child := NULLPTR) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff08 \rightarrow Next := ff04) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff12 \rightarrow Next := ff08) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff16 \rightarrow NEXT := ff12) \rightarrow Stack\ Frame$
$destroyNodeRecursively(head := ff16) \rightarrow Stack\ Frame$

And, it will exit the current function call.

$destroyNodeRecursively(Node * node: NULLPTR)$

$doesn't\ exit\ from\ the\ function[As, it\ is\ recursive].$

```
    destroyNodeRecursively(node->next);
```

$destroyNodeRecursively(node: ff04 \rightarrow Next: NULLPTR)$

$\rightarrow PUSH\ NULLPTR\ to\ the\ STACK.$

$destroyNodeRecursively(node: ff04 \rightarrow Next := NULLPTR) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff08 \rightarrow Next := ff04) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff12 \rightarrow Next := ff08) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff16 \rightarrow NEXT := ff12) \rightarrow Stack\ Frame$
$destroyNodeRecursively(head := ff16) \rightarrow Stack\ Frame$

$Now\ ,it\ will\ assign, as\ it\ call\ recursively:$

$destroyNodeRecursively(Node * node) =$

$destroyNodeRecursively(node \rightarrow child: NULLPTR\ )$

$i.e. Node * node = node \rightarrow child: NULLPTR$

$And, it\ is\ the\ base\ case:$

```
    if (node == nullptr)
        return;
```

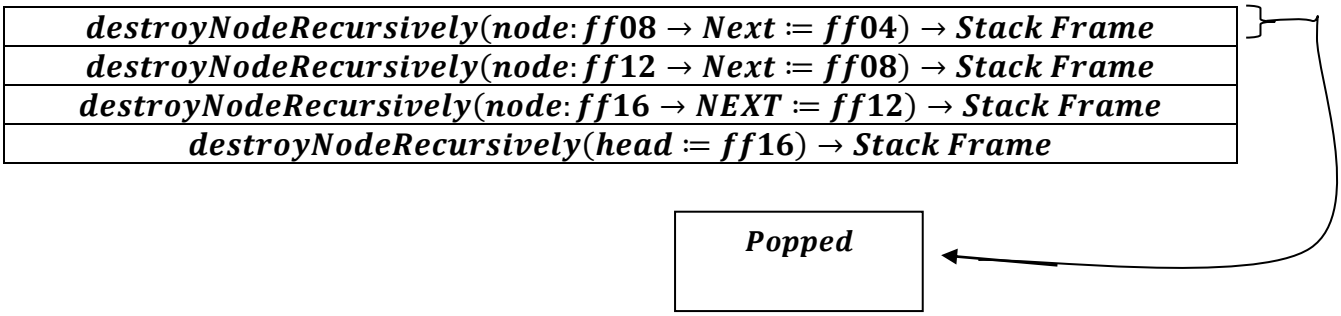
$As\ it\ goes\ to\ base\ case\ ,return\ ;hence\ pop\ the\ current\ stack\ frame\ for\ nullptr\ pop\ up\ immediately\ .$

$destroyNodeRecursively(node: ff04 \rightarrow Next := NULLPTR) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff08 \rightarrow Next := ff04) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff12 \rightarrow Next := ff08) \rightarrow Stack\ Frame$
$destroyNodeRecursively(node: ff16 \rightarrow NEXT := ff12) \rightarrow Stack\ Frame$
$destroyNodeRecursively(head := ff16) \rightarrow Stack\ Frame$

So both recursive calls return immediately $\begin{bmatrix} \textbf{Base Case Met} \\ \textbf{for both the calls} \\ \textbf{i.e. NULLPTR} \end{bmatrix}$.

```
    // 1. Destroy Child List first
    destroyNodeRecursively(node->child);

    // 2. Destroy Next List
    destroyNodeRecursively(node->next);
```

This is the time , when the deeper recursive calls finish , Now it ready for unwinding.

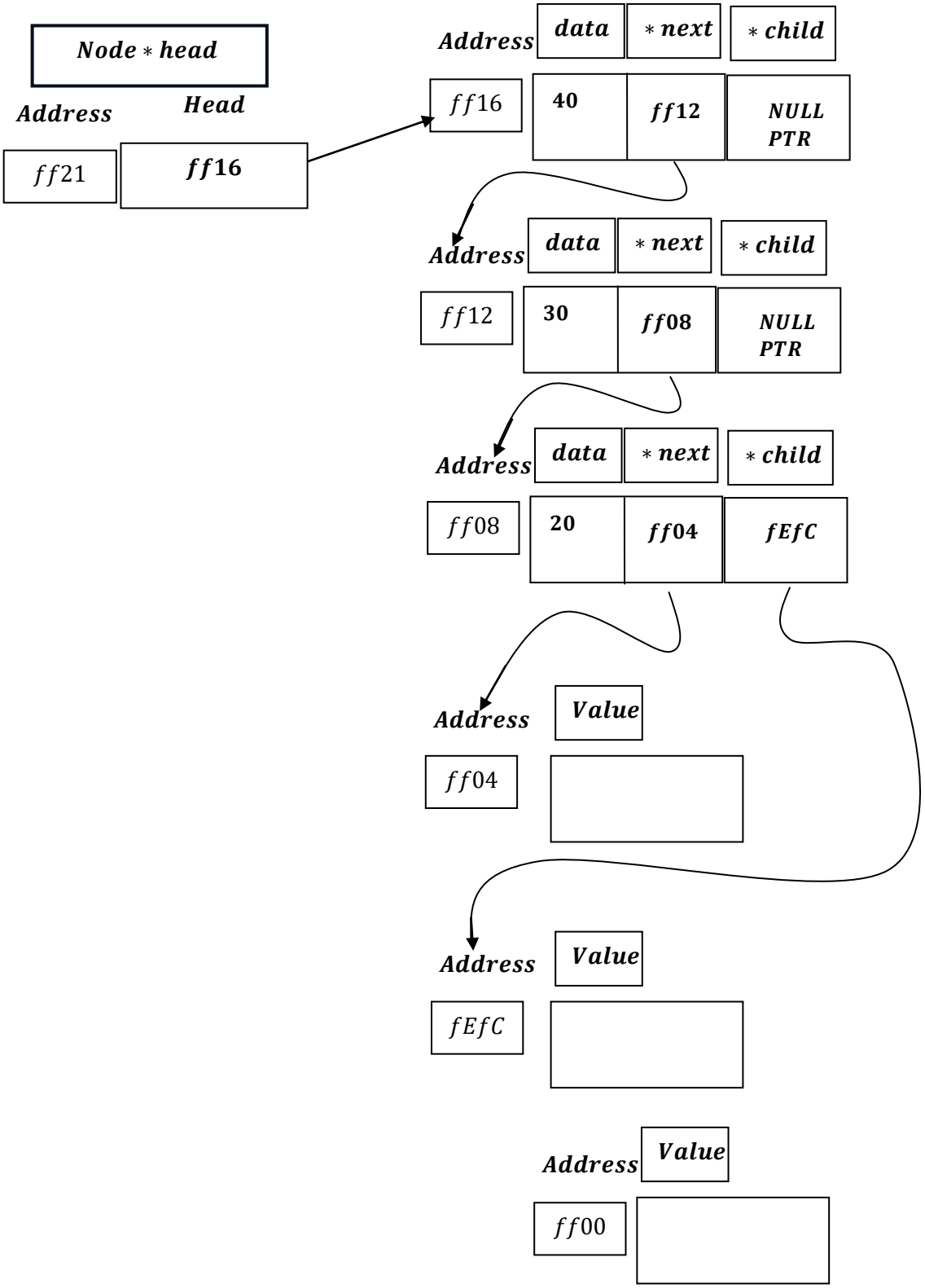


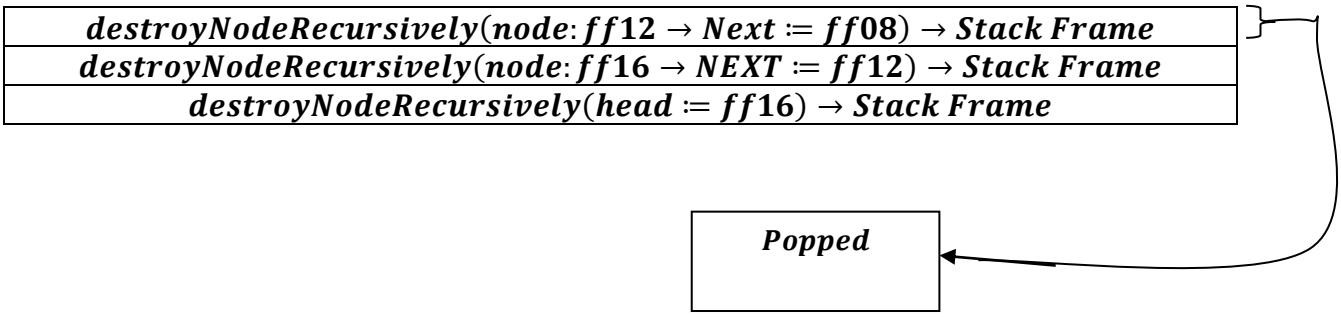
And it will run:

```
    cout << "Freeing node: " << node->data << endl;
    free(node);
```

Print : ``***Freeing Node***:`` [***node***: $\boldsymbol{ff04}$ → ***data***] = **10** .

Free(***node***: $\boldsymbol{ff04}$)


Node * head
Address
Head
ff21
ff16
Address
data
* next
* child
ff16
40
ff12
NULL PTR
Address
data
* next
* child
ff12
30
ff08
NULL PTR
Address
data
* next
* child
ff08
20
ff04
fEfC
Address
Value
ff04
Address
Value
fEfC
Address
Value
ff00

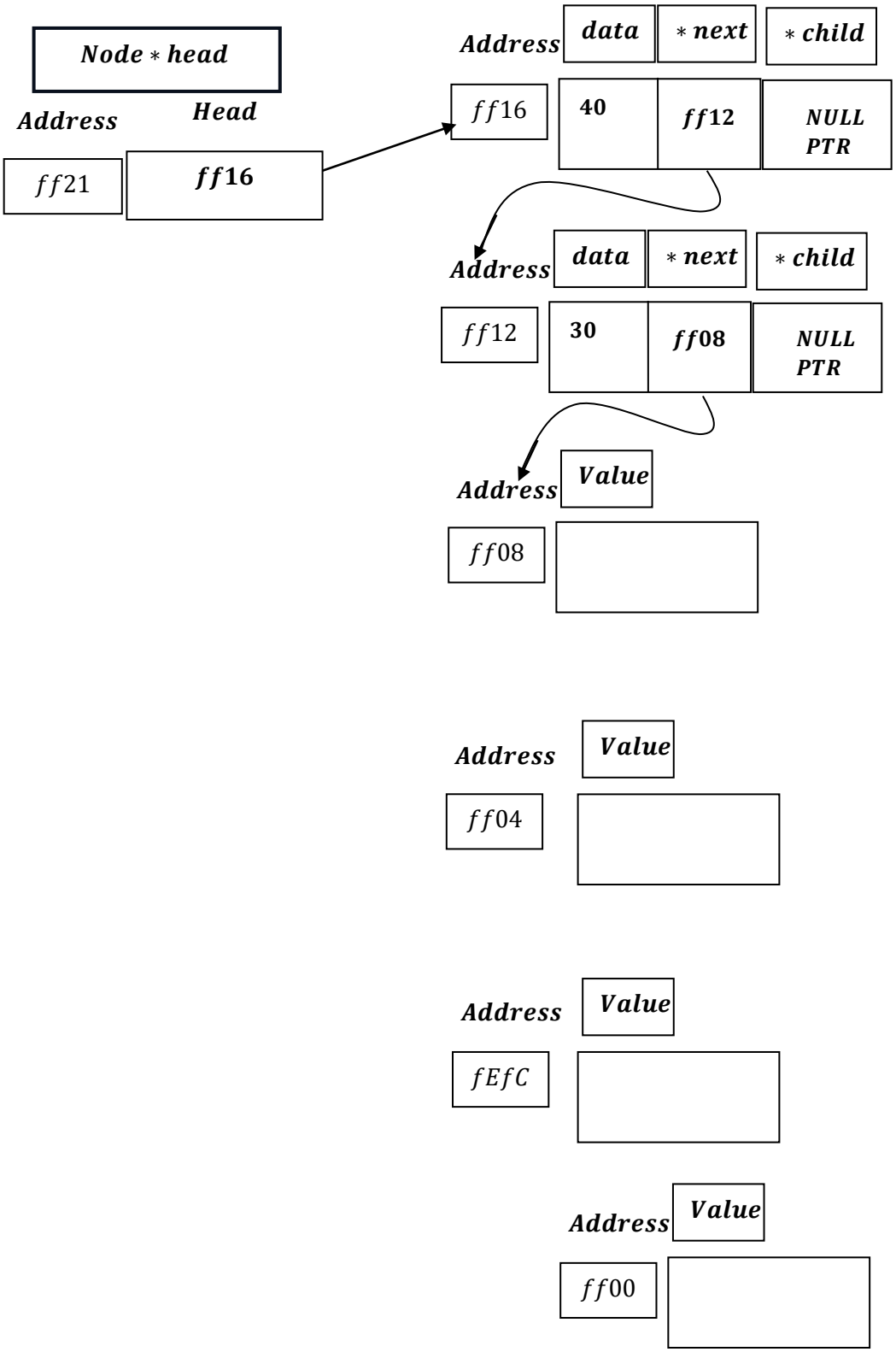


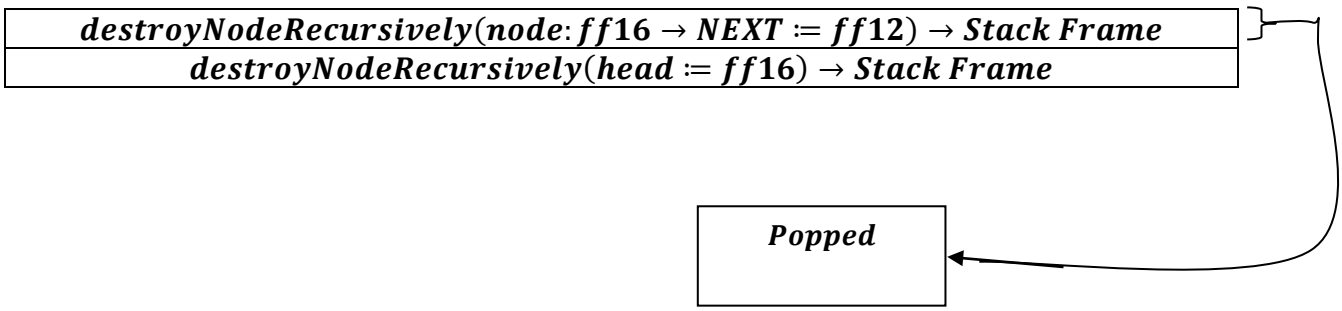
And it will run:

```
    cout << "Freeing node: " << node->data << endl;
    free(node);
```

Print : ``***Freeing Node***:`` [***node***: ***ff08*** → ***data***] = **20** .

Free(***node***: ***ff08***)


Node * head
Address
Head
ff21
ff16
Address
data
* next
* child
ff16
40
ff12
NULL PTR
Address
data
* next
* child
ff12
30
ff08
NULL PTR
Address
Value
ff08
Address
Value
ff04
Address
Value
fEfC
Address
Value
ff00

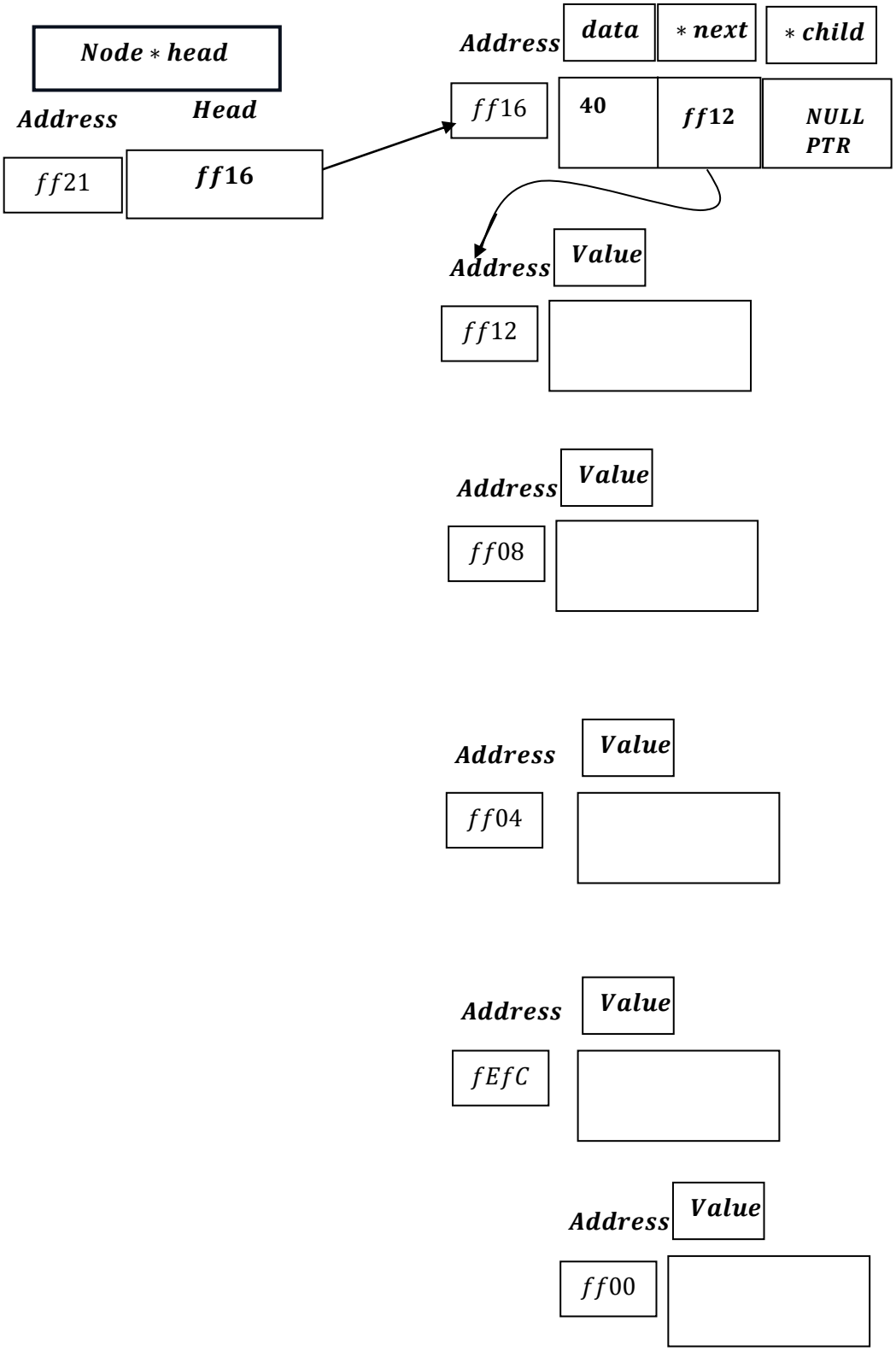


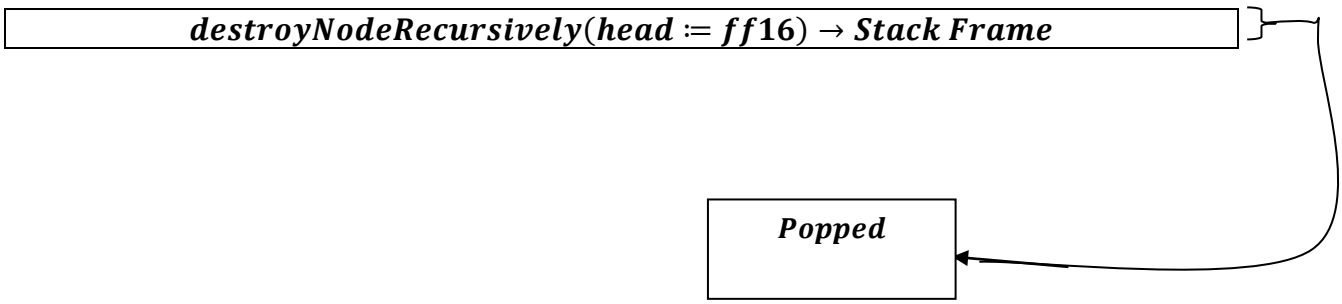
And it will run:

```
    cout << "Freeing node: " << node->data << endl;
    free(node);
```

Print : ``***Freeing Node***:`` [***node***: ***ff12*** → ***data***] = **30** .

Free(***node***: ***ff12***)


Node * head
Address
Head
ff21
ff16
Address
data
* next
* child
ff16
40
ff12
NULL PTR
Address
Value
ff12
Address
Value
ff08
Address
Value
ff04
Address
Value
fEfC
Address
Value
ff00

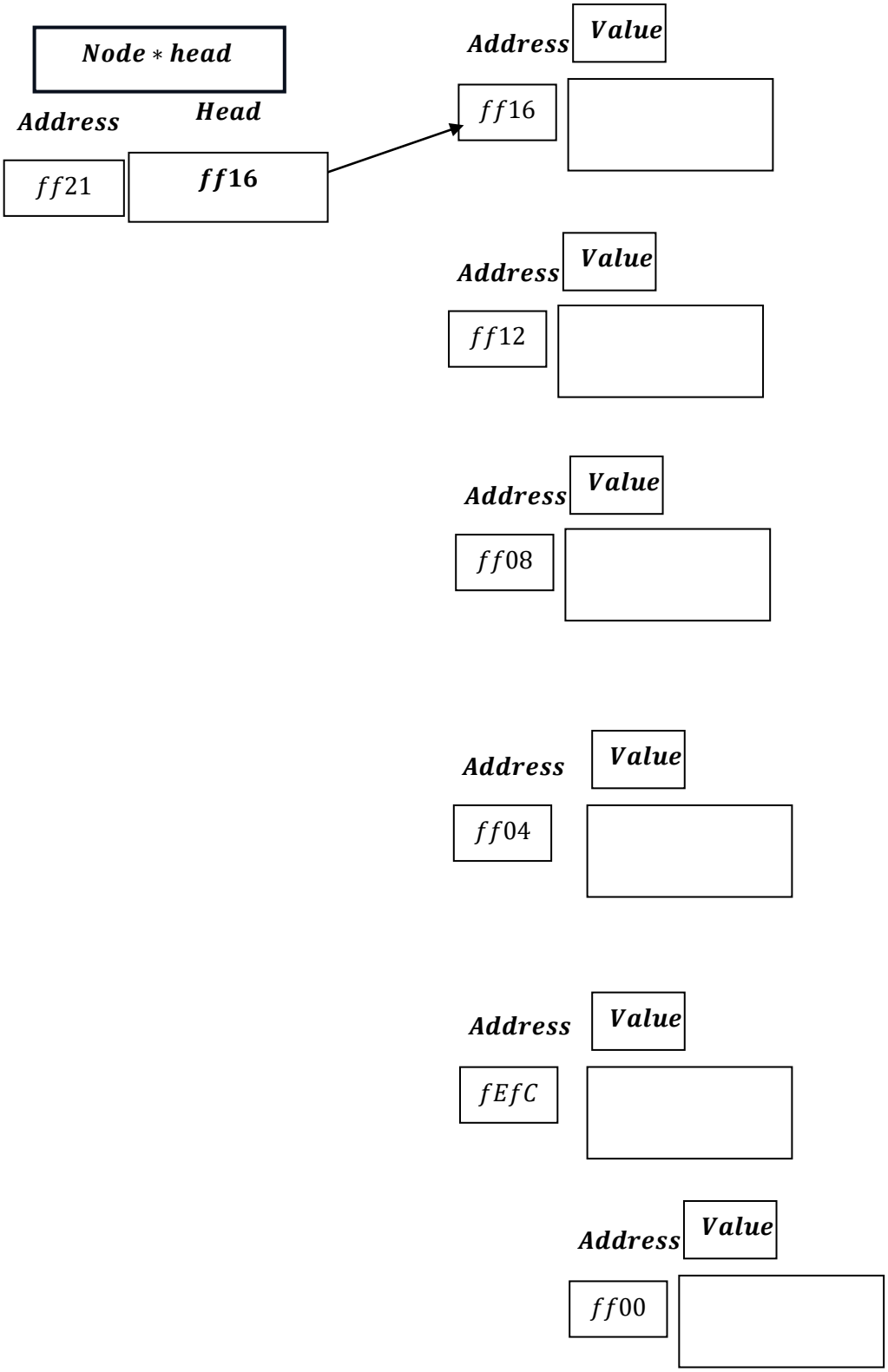


And it will run:

```
    cout << "Freeing node: " << node->data << endl;
    free(node);
```

$\boldsymbol{Print}$: ``$\boldsymbol{Freeing\ Node}$:`` $[\boldsymbol{node}: \boldsymbol{ff16} \rightarrow \boldsymbol{data}] = \mathbf{40}$.

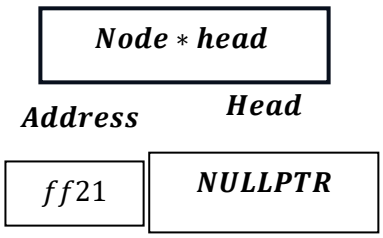
$\boldsymbol{Free}(\boldsymbol{node}: \boldsymbol{ff16})$


Node * head
Address
Head
ff21
ff16
Address
Value
ff16
Address
Value
ff12
Address
Value
ff08
Address
Value
ff04
Address
Value
fEfC
Address
Value
ff00

Now, coming back to :

```
void destroyList()
{
    destroyNodeRecursively(head);
    head = nullptr;
    cout << "Entire list destroyed." << endl;
}
```

Head = NULLPTR



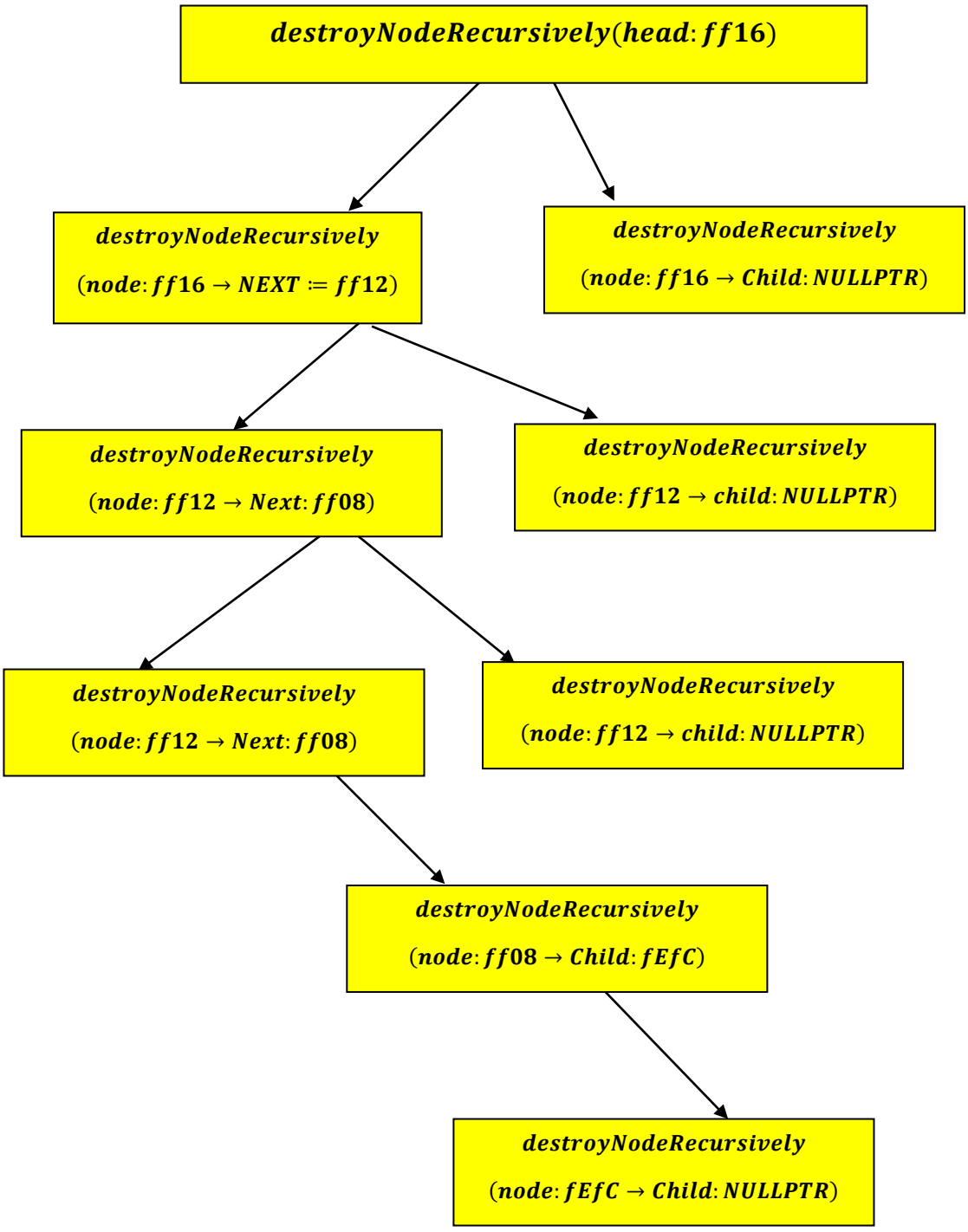
Now , list becomes empty.

```
    cout << "Entire list destroyed." << endl;
```

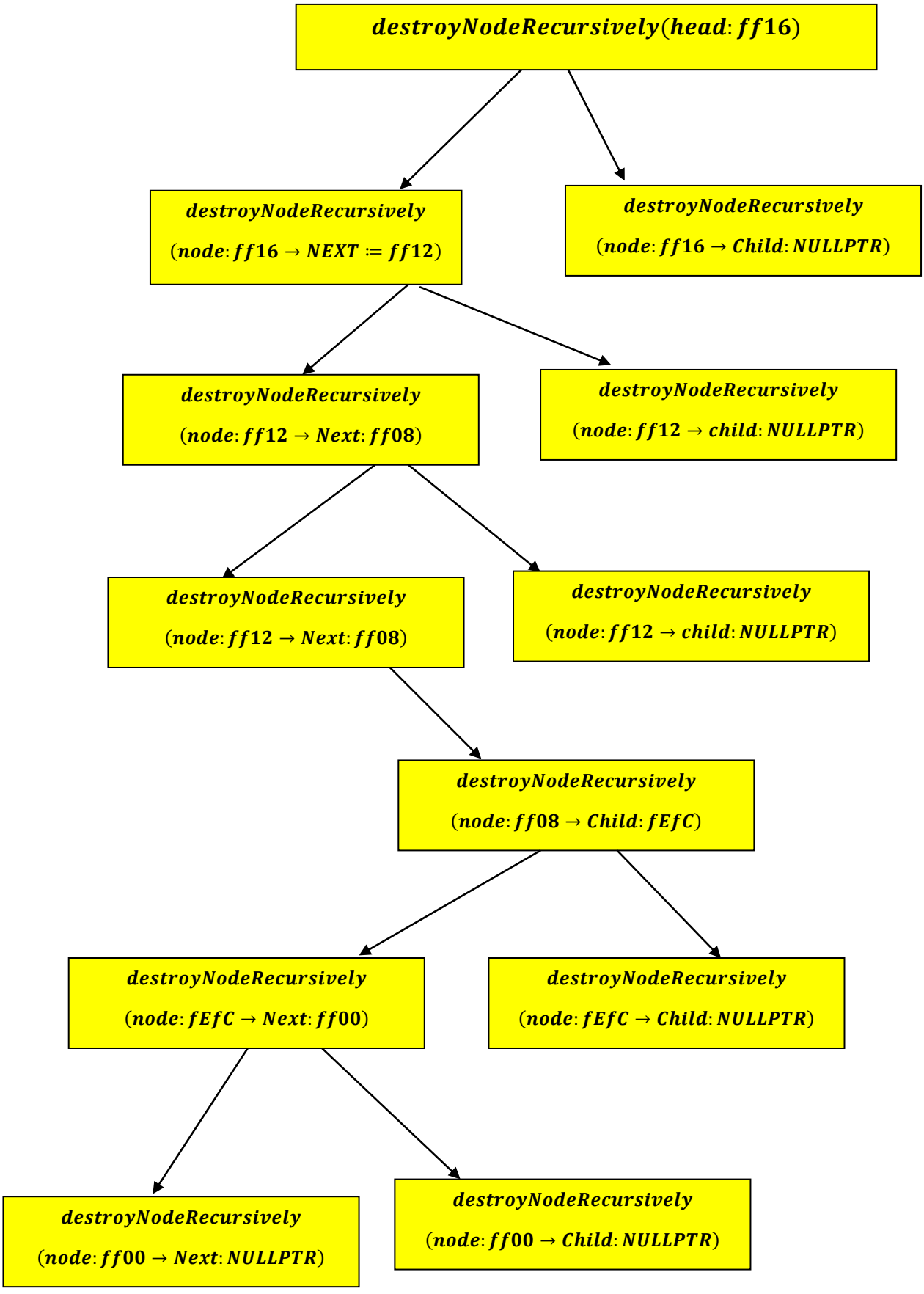
i.e. print ``Entire List destroyed.``

---

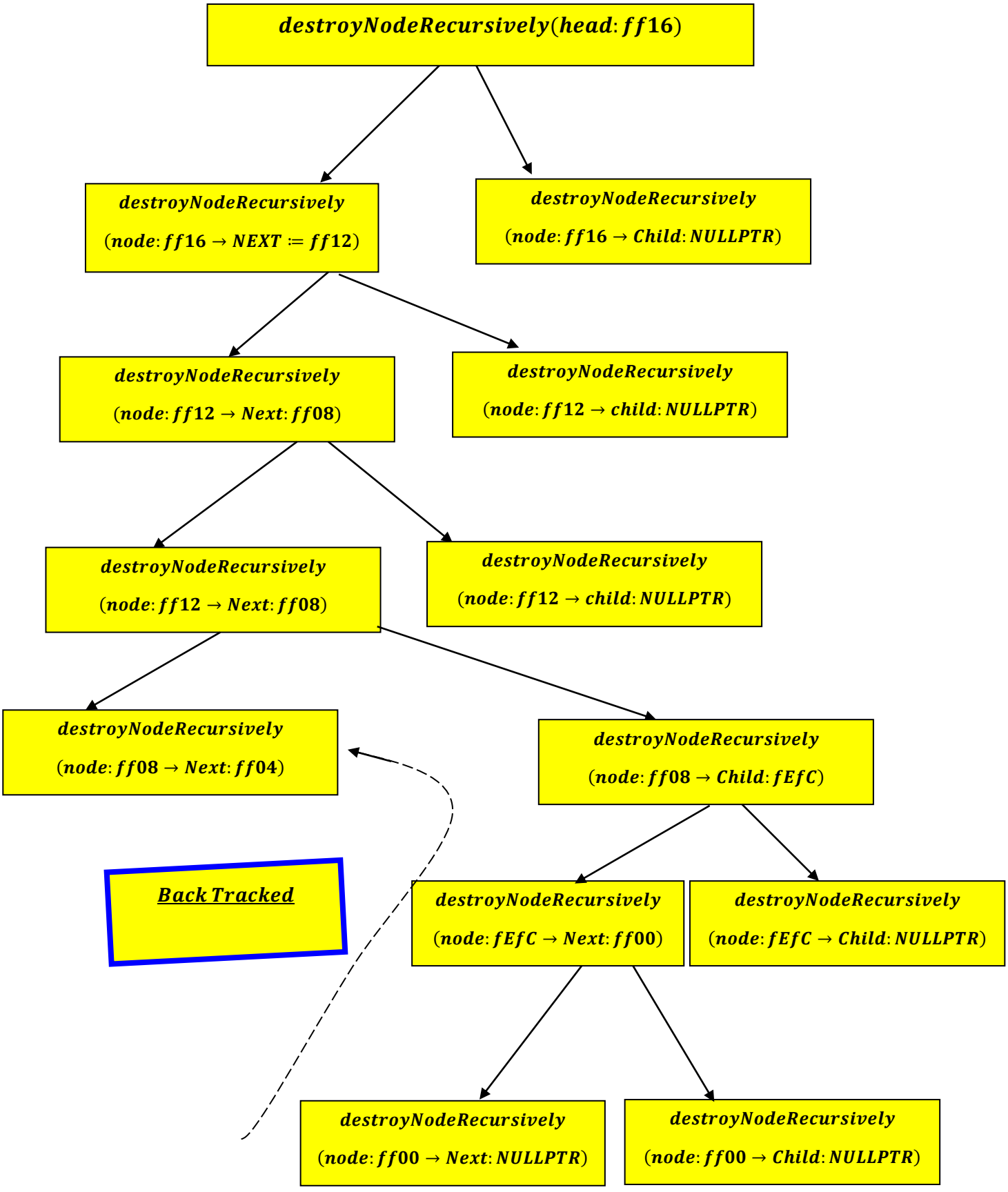
if we see the recursive tree then:

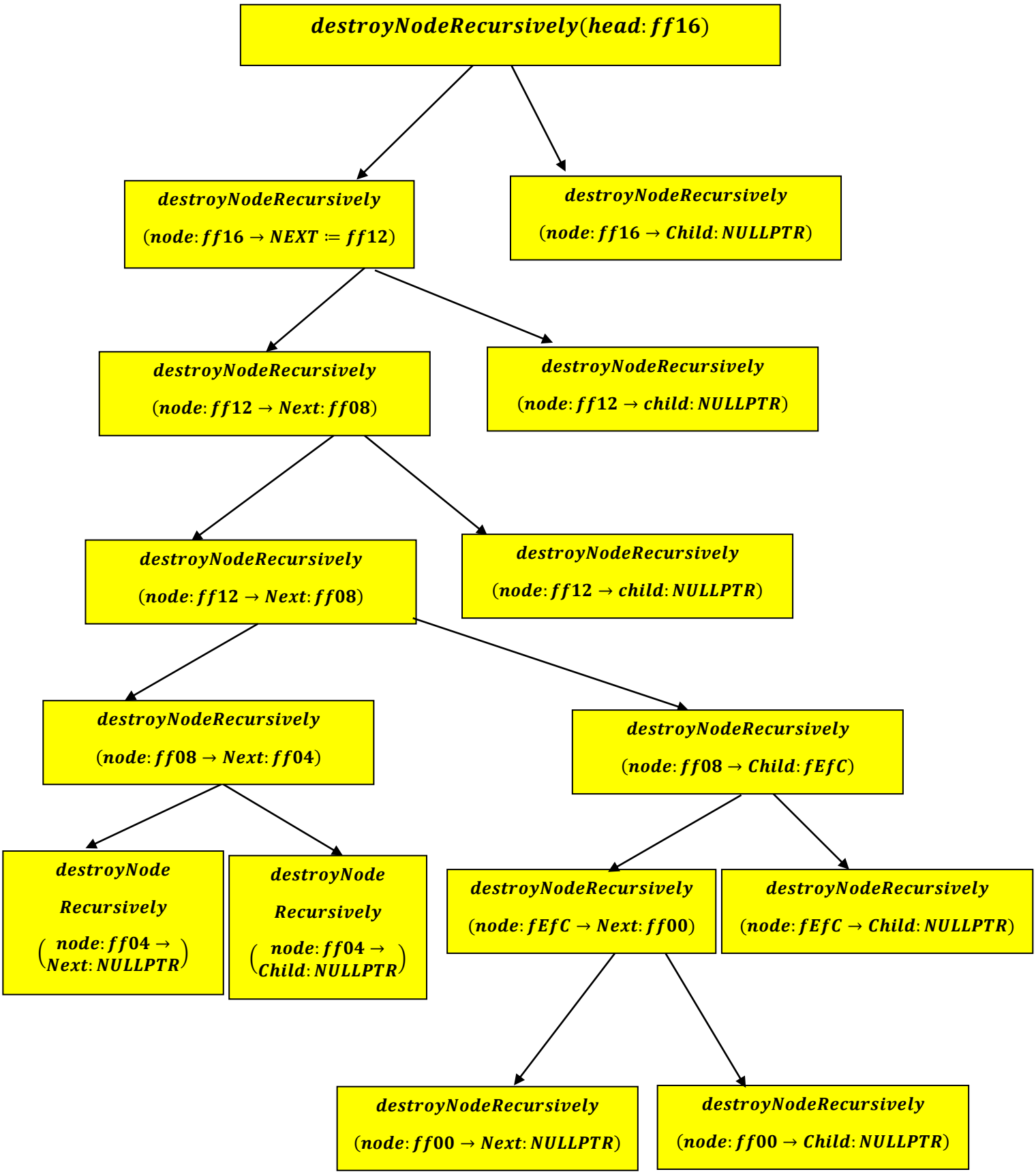


By Back Tracking again we get:


destroyNodeRecursively(head: ff16)
destroyNodeRecursively (node: ff16 → NEXT := ff12)
destroyNodeRecursively (node: ff16 → Child: NULLPTR)
destroyNodeRecursively (node: ff12 → Next: ff08)
destroyNodeRecursively (node: ff12 → child: NULLPTR)
destroyNodeRecursively (node: ff12 → Next: ff08)
destroyNodeRecursively (node: ff12 → child: NULLPTR)
destroyNodeRecursively (node: ff08 → Child: fEfC)
destroyNodeRecursively (node: fEfC → Next: ff00)
destroyNodeRecursively (node: fEfC → Child: NULLPTR)
destroyNodeRecursively (node: ff00 → Next: NULLPTR)
destroyNodeRecursively (node: ff00 → Child: NULLPTR)

After unwinding and poppping , it will again backtrack to:




destroyNodeRecursively(head: ff16)
destroyNodeRecursively (node: ff16 → NEXT ≔ ff12)
destroyNodeRecursively (node: ff16 → Child: NULLPTR)
destroyNodeRecursively (node: ff12 → Next: ff08)
destroyNodeRecursively (node: ff12 → child: NULLPTR)
destroyNodeRecursively (node: ff12 → Next: ff08)
destroyNodeRecursively (node: ff12 → child: NULLPTR)
destroyNodeRecursively (node: ff08 → Next: ff04)
destroyNodeRecursively (node: ff08 → Child: fEfC)
destroyNode Recursively (node: ff04 → Next: NULLPTR)
destroyNode Recursively (node: ff04 → Child: NULLPTR)
destroyNodeRecursively (node: fEfC → Next: ff00)
destroyNodeRecursively (node: fEfC → Child: NULLPTR)
destroyNodeRecursively (node: ff00 → Next: NULLPTR)
destroyNodeRecursively (node: ff00 → Child: NULLPTR)